UNIVERSITY OF CAMBRIDGE

Multicore Semantics and Programming - Assignment 2

Author

Kyle Fram - kdf25

Lecturers

Professors Tim Harris and Peter Sewell

March 24, 2026

Contents

1 **1**
1.a (2 hours) . . . 1
1.b (30 min) . . . 1
1.c (2 min) . . . 2
1.d (30 min) . . . 2
1.e (20 min) . . . 2

2 **3**
2.a (45 min) . . . 3
2.b (10 min) . . . 3
2.c (20 min) . . . 3

3 **4**
3.a (45 min) . . . 4
3.b (3 hrs) . . . 4
3.c (5 min) . . . 5
3.d (10 min) . . . 7
3.e (15 min) . . . 8

4 **9**
4.a (2hr) . . . 9
4.b (20min) . . . 9
4.c (1.5 hr) . . . 10
4.d (1 min) . . . 10
4.e (5 min) . . . 11

1 Appendix **12**
1.a diy7 . . . 12
1.b litmus7 . . . 12
1.c herd7 . . . 20
1.d rmem . . . 21
1.e mcompare7 . . . 23

2 Appendix **25**
2.a . . . 25

3 Appendix **26**

References **29**

List of Figures **I**

List of Tables **II**

UNIVERSITY OF CAMBRIDGE

I have written the estimated time for each part in parentheses under subsection label. Generally, setting up diy and opam, as well as relearning a bit of C, made up the lion's share of the time.

1

1.a (2 hours)

We use these commands to generate the tests:

```
diyone7 -arch AArch64 "PodWR Fre PodWR Fre" -name SB
diyone7 -arch AArch64 "PodWR Fre PodWR Fre PodWR Fre" -name SB3
diyone7 -arch AArch64 "DMB.SYdWR Fre DMB.SYdWR Fre" -name SB+fen
diyone7 -arch AArch64 "DMB.SYdWR Fre DMB.SYdWR Fre DMB.SYdWR Fre" -name
    SB3+fen
```

'SB3' is my shorthand for Store Buffering with 3 threads. I am on an M4 Max chip (AArch64), and I expect to observe significant relaxations in the unfenced SB and SB3 tests.

Listing 1: System Specs

```
Hardware:

    Hardware Overview:

      Model Name: MacBook Pro
      Model Identifier: Mac16,5
      Model Number: MX303B/A
      Chip: Apple M4 Max
      Total Number of Cores: 14 (10 performance and 4 efficiency)
      Memory: 36 GB
      System Firmware Version: 11881.81.4
      OS Loader Version: 11881.81.4
      Serial Number (system): C00C3JNQFV
      Hardware UUID: 6B16ED3A-820B-5B1F-A79E-0B45691B2525
      Provisioning UDID: 00006041-000849C90204801C
      Activation Lock Status: Enabled
```

1.b (30 min)

Running with litmus7, we notice 7.7 and 9.4 seconds to run SB with and without barrier respectively. As expecteed, the test condition is not validated in the fenced case, but is in the unfenced case.

For SB3, we see 10.4 and 12.4 seconds respectively, a slightly higher percentage difference (81% vs 83%). We see the expected fenced vs unfenced results.

Another curious difference is that while unfenced SB has only 234 out of 1e8 iterations validating the test condition, SB3 has 1870409 out of 1e8 iterations validating the test condition. We may see more prevalent compiler optimization and reordering in the latter case with more threads, possibly the extra thread results in more optimization opportunities.

Using -s 1k, -r 100k to get 1e8 for each case:

```
litmus7 SB.litmus -s 1k -r 100k > logs/q1/b_1.txt
litmus7 SB+fen.litmus -s 1k -r 100k > logs/q1/b_1fen.txt
litmus7 SB3.litmus -s 1k -r 100k > logs/q1/b_2.txt
litmus7 SB3+fen.litmus -s 1k -r 100k > logs/q1/b_2fen.txt
```

1.c (2 min)

Comparing the axiomatic results, we naturally notice no difference in timing. However we notice the expected difference in the results, with our test condition missing from possible outputs in the fenced cases.

```
herd7 SB.litmus > logs/q1/c_1.txt
herd7 SB3.litmus > logs/q1/c_1fen.txt
herd7 SB+fen.litmus > logs/q1/c_2.txt
herd7 SB3+fen.litmus > logs/q1/c_2fen.txt
```

1.d (30 min)

We use interactive -false and eager -true flags. In both cases, the fenced version notes 'allowed not found' agreeing with prior analaysis.

```
rmem -model flat -interactive false -eager true SB.litmus > logs/q1/d_1.
    txt
rmem -model flat -interactive false -eager true SB+fen.litmus > logs/q1/
    d_1fen.txt
rmem -model flat -interactive false -eager true SB3.litmus > logs/q1/d_2
    .txt
rmem -model flat -interactive false -eager true SB3+fen.litmus > logs/q1
    /d_2fen.txt
```

1.e (20 min)

Again, we reinforced the expected results from the above.

```
mcompare7 logs/q1/b_1.txt logs/q1/c_1.txt > logs/q1/e_1.txt
mcompare7 logs/q1/b_1fen.txt logs/q1/c_1fen.txt > logs/q1/e_1fen.txt
mcompare7 logs/q1/b_2.txt logs/q1/c_2.txt > logs/q1/e_2.txt
mcompare7 logs/q1/b_2fen.txt logs/q1/c_2fen.txt > logs/q1/e_2fen.txt
```

2

2.a (45 min)

Forbidden by ARM ARM. Consider B.2.3.2. The important note here is in the Note box: "This applies even in cases where the Intrinsic Data Dependency might be perceived to be a false dependency." Further, this is a 'data dependency' as defined by B.2.3.6, and since this architecture respects data dependencies and coherence, the memory accesses must follow in strict order, and therefore $x = 3$ is forbidden.

2.b (10 min)

RISC-V as per 17.1.1.3, rule 10 enforces PPO over data dependencies, such that the $d : y = 1$ read and corresponding $e : x = 1$ write must be ordered after the $x = 3$ write, and thus according to the Load Value Axiom, would be 'latest' in global memory order and therefore $x = 3$ must not be reachable in practice.

2.c (20 min)

As per the above, on AArch64, the test is forbidden and we see axiomatically this is reinforced. We also ran 1e8 litmus tests and saw no positive witnesses, enforcing our result empirically.

```
    diyone7 -arch AArch64 "Wse DMB.SYdWW Rfe DpDatadW" -name sub
    rmem -model flat -interactive false -eager true sub.litmus > logs/q2/c_1
        .txt
    herd7 sub.litmus > logs/q2/c.txt
```

```
Test S+fen+data-wsi Allowed
States 3
1:X1=0; 1:X3=0;
1:X1=0; 1:X3=1;
1:X1=1; 1:X3=1;
No
Witnesses
Positive: 0 Negative: 3
Condition exists (1:X1=1 /\ 1:X3=0)
Observation S+fen+data-wsi Never 0 3
Time S+fen+data-wsi 0.01
Hash=6cd45049c39f769e3371a253e6fa3e5c
```

3

3.a (45 min)

Consider a 'fence' operation. This provides a 'synchronization point,' such that all memory operations before and after the fence wait for all others, and once all operations meet at the fence, they tear it down. Now, consider propping a ladder up on one side of the metaphorical fence. For an acquire or release, this synchronization is 'one-way,' so anyone on the far side cannot access until the last operation throws the ladder over the fence.

3.b (3 hrs)

I implemented Message Passing with pthreads and acquire/release semantics. This program has a writer thread which writes two values in succession with a release between the writes, and then a reader thread which reads those respectively with an acquire fence between them. Pthreads only allows one arg so I wrapped the values in a struct. The release/acquire semantics ensure that x=1,y=1 always holds at termination, but does not force the writer to wait for respective read.

Listing 2: MessagePassing

```
#include <stdio.h>
#include <pthread.h>
#include <stdatomic.h>

struct targs {
    int *x;
    atomic_int *y;      // syncer
};

void* writer(void *arg) {
    struct targs *args = (struct targs *)arg;

    *(args->x) = 1; // write

    // release
    atomic_store_explicit(args->y, 1, memory_order_release);
    return NULL;
}

void* reader(void *arg) {
    struct targs *args = (struct targs *)arg;

    // acquire
    while (atomic_load_explicit(args->y, memory_order_acquire) == 0) {}
```

```c

    // read
    printf("read x: %d\n", *(args->x));
    return NULL;
}

int main(void) {
    pthread_t t1, t2;
    int x = 0;
    atomic_int y = 0;

    struct targs args = {&x, &y};

    pthread_create(&t1, NULL, writer, &args);
    pthread_create(&t2, NULL, reader, &args);

    pthread_join(t1, NULL);
    pthread_join(t2, NULL);

    return 0;
}
```

3.c (5 min)

In a certain sense, this is consistent, we have DMB ISH for the release and DMB ISHLD for the acquire. In another sense, this is redundant, as we are using atomics and the compiler uses stlr and ldar, which enforce sequential consistency anyhow. Overall this is indeed consistent, albeit redundant.
Here is the compiled code (full compilation in appendix):

```asm
reader:
        sub     sp, sp, #48
        stp     x29, x30, [sp, #32]
        add     x29, sp, #32
        stur    x0, [x29, #-8]
        ldur    x8, [x29, #-8]
        str     x8, [sp, #16]
        b       .LBB1_1
```

```
writer:
        sub     sp, sp, #32
        str     x0, [sp, #24]
        ldr     x8, [sp, #24]
        str     x8, [sp, #16]
        ldr     x8, [sp, #16]
        ldr     x9, [x8]
        mov     w8, #1
        str     w8, [x9]
        ldr     x9, [sp, #16]
        ldr     x9, [x9, #8]
        str     w8, [sp, #12]
        ldr     w8, [sp, #12]
        stlr    w8, [x9]
        mov     x0, xzr
        add     sp, sp, #32
        ret
```


UNIVERSITY OF CAMBRIDGE

```
main:
        sub     sp, sp, #96
        stp     x29, x30, [sp, #80]
        add     x29, sp, #80
        mov     w8, wzr
        str     w8, [sp, #28]
        stur    wzr, [x29, #-4]
        sub     x9, x29, #28
        stur    wzr, [x29, #-28]
        sub     x8, x29, #32
        stur    wzr, [x29, #-32]
        add     x3, sp, #32
        str     x3, [sp, #8]
        str     x9, [sp, #32]
        str     x8, [sp, #40]
        sub     x0, x29, #16
        mov     x1, xzr
        str     x1, [sp, #16]
        adrp    x2, writer
        add     x2, x2, :lo12:writer
        bl      pthread_create
        ldr     x3, [sp, #8]
        ldr     x1, [sp, #16]
        sub     x0, x29, #24
        adrp    x2, reader
        add     x2, x2, :lo12:reader
        bl      pthread_create
        ldr     x1, [sp, #16]
        ldur    x0, [x29, #-16]
        bl      pthread_join
        ldr     x1, [sp, #16]
        ldur    x0, [x29, #-24]
        bl      pthread_join
        ldr     w0, [sp, #28]
        ldp     x29, x30, [sp, #80]
        add     sp, sp, #96
        ret
```

3.d (10 min)

Litmus test of the above:

Listing 3: MP with Release/Acquire Litmus

```
AArch64 MP-RAQ
"MessagePassingAccReq"
{
0:X1=x; 0:X2=y;
1:X1=x; 1:X2=y;
}
```

```
 P0           | P1           ;
 MOV W0,#1    | LDAR W0,[X2] ;
 STLR W0,[X1] | DMB ISHLD    ;
 DMB ISH      | LDR W1,[X1]  ;
 STLR W0,[X2] |              ;
exists
(1:X0=1 /\ 1:X1=0)
```

3.e (15 min)

As above, we expect and see 'forbidden' here, we have this doubly redundant synchronization with fences and store-releases.

```
    cd ctests
    herd7 ../mpraq.litmus > ../logs/q3/e.txt
```

Listing 4: Litmus Results (herd7)

```
Test MP-RAQ Allowed
States 3
1:X0=0; 1:X1=0;
1:X0=0; 1:X1=1;
1:X0=1; 1:X1=1;
No
Witnesses
Positive: 0 Negative: 3
Flag Assuming-common-inner-shareable-domain
Condition exists (1:X0=1 /\ 1:X1=0)
Observation MP-RAQ Never 0 3
Time MP-RAQ 0.01
Hash=e1d00fba97008f6a98e58e7b0231e4ec
```

4

4.a (2hr)

We define *ppo* to be the communication relation without the 'writes to the same location' relation as a relaxation of x86-TSO.

```
TSOREL
(*relaxed total store order — branch from sc.cat by removing specific writes
    and reads *)
(* Define the co and fr relation *)
include "cos.cat"

(* Group communication relations together *)
let com = rf | fr | co
(* writes to the same loc, and writes to reads *)
let wr = po & (W * R)
let ww = (po & (W*W)) \ loc
let ppo = po \ (wr | ww)

(* Sequential consistency condition *)
acyclic ppo | com as sc
```

4.b (20min)

S is a good test to verify our model due to its two writes to different memory locations. We expect different results in our relaxed model vs. the standard x86-TSO model.

```

    diyone7 —arch AArch64 PodWW Rfe PodRW Wse —name S
    herd7 S.litmus —model cats/tsorel.cat>> logs/q4/b1.txt
    herd7 S.litmus > logs/q4/b2.txt
```

Indeed we see that in the relaxed model, we sometimes see the 'reordered' outcome, where in TSO we never see this.

Listing 5: Litmus Results (herd7) (PPO Model)

```
Test S Allowed
States 4
1:EAX=0; [x]=1;
1:EAX=0; [x]=2;
1:EAX=1; [x]=1;
1:EAX=1; [x]=2;
Ok
Witnesses
```

```
Positive: 1 Negative: 3
Condition exists ([x]=2 /\ 1:EAX=1)
Observation S Sometimes 1 3
Time S 0.00
Hash=dc4f01f21e6e1265aeb6ccda34ba3e4c
```

Listing 6: Litmus Results (herd7) (x86 TSO Model)

```
Test S Allowed
States 3
1:EAX=0; [x]=1;
1:EAX=0; [x]=2;
1:EAX=1; [x]=1;
No
Witnesses
Positive: 0 Negative: 3
Condition exists ([x]=2 /\ 1:EAX=1)
Observation S Never 0 3
Time S 0.00
Hash=dc4f01f21e6e1265aeb6ccda34ba3e4c
```

4.c (1.5 hr)

Here is the paper math definition of the relaxed model based on the x86-tso model referenced in [1] and the notes. I have removed MFENCE and LOCK for clarity. **RB** and **RM** are unchanged, we include **WB** definitions and keep the model from the notes otherwise unchanged. Crucially, we partition the write buffer into a 'skipped' and a 'rest' queue to allow for arbitrary reordering of writes to different locations, but we still require that writes to the same location are ordered.

$$
\begin{array}{rcl}
\text{Events} \quad e & | & a : t : \mathrm{W}\ x = v \\
 & | & a : t : \mathrm{R}\ x = v \\
 & | & a : t : \mathrm{Dw}\ x = v \\
\text{Memory} \quad m & | & \langle M : \text{addr} \to \text{value} \\
 & | & B : \text{tid} \to \text{write_event list} \rangle \\
\text{Operations} \quad o & | & \mathbf{WB} : m.B(t) = b_{skipped} \mathbin{@} b_{rest} \quad \forall (\ldots \mathrm{W}\ y = v') \in b_{skipped}.\, y \neq x \\
 & | & \quad m \xrightarrow{a:t:\mathrm{W}\ x=v} m \oplus \langle B := m.B \oplus (t \mapsto (b_{skipped} \mathbin{@} [a : t : \mathrm{W}\ x = v] \mathbin{@} b_{rest})) \rangle
\end{array}
$$

4.d (1 min)

C allows for this sort of relaxation natively with stdatomic and memory_order_relaxed. If these are executed with memory_order_relaxed on ARM for example, two writes to

different locations may be reordered. If we wanted to use C 'as it is expected,' the compiler would need to enforce fencing between writes to ensure sequential consistency.

4.e (5 min)

Arbitrarily reordering of reads from different locations may be more tricky. Take Message Passing for example, arbitrary reordering of reads of different locations effectively breaks any notion of intrathread communication, and fencing, reliance on atomicity, or at least acquire/release must be used often. Realistically, a language without fences, barriers, or sequential reads becomes chaos.

UNIVERSITY OF CAMBRIDGE

1 Appendix

1.a diy7

1.b litmus7

Listing 7: SB.litmus

```
%%%%%%%%%%%%%%%%%%%%%%%%%
% Results for SB.litmus %
%%%%%%%%%%%%%%%%%%%%%%%%%
AArch64 SB
"PodWR Fre PodWR Fre"

{
 0:X1=x; 0:X2=y;
 1:X1=y; 1:X2=x;
}
 P0          | P1          ;
 MOV W0,#1   | MOV W0,#1   ;
 STR W0,[X1] | STR W0,[X1] ;
 LDR W3,[X2] | LDR W3,[X2] ;

exists (0:X3=0 /\ 1:X3=0)
Generated assembler
        ;START _litmus_P0
        mov     w0, #1                          ; =0x1
        str     w0, [x15]
        ldr     w17, [x16]
        ;START _litmus_P1
        mov     w0, #1                          ; =0x1
        str     w0, [x15]
        ldr     w17, [x16]

Test SB Allowed
Histogram (4 states)
234   *>0:X3=0; 1:X3=0;
49999841:>0:X3=1; 1:X3=0;
49999909:>0:X3=0; 1:X3=1;
16    :>0:X3=1; 1:X3=1;
Ok

Witnesses
Positive: 234, Negative: 99999766
```

```
Condition exists (0:X3=0 /\ 1:X3=0) is validated
Hash=0e1eb22a42a7de6af162a7e5cbb4e4d3
Generator=diyone7 (version 7.58)
Com=Fr Fr
Orig=PodWR Fre PodWR Fre
Observation SB Sometimes 234 99999766
Time SB 7.70

Machine:Kyles-MacBook-Pro.local
Hardware:

    Hardware Overview:

      Model Name: MacBook Pro
      Model Identifier: Mac16,5
      Model Number: MX303B/A
      Chip: Apple M4 Max
      Total Number of Cores: 14 (10 performance and 4 efficiency)
      Memory: 36 GB
      System Firmware Version: 11881.81.4
      OS Loader Version: 11881.81.4
      Serial Number (system): C00C3JNQFV
      Hardware UUID: 6B16ED3A-820B-5B1F-A79E-0B45691B2525
      Provisioning UDID: 00006041-000849C90204801C
      Activation Lock Status: Enabled

Revision exported, version 7.58
Command line: litmus7 SB.litmus -s 1k -r 100k
Parameters
#define SIZE_OF_TEST 1000
#define NUMBER_OF_RUN 100000
#define AVAIL 1
#define STRIDE (-1)
#define MAX_LOOP 0
/* gcc options: -Wall -std=gnu99 -O2 */
/* barrier: user */
/* launch: changing */
/* affinity: none */
/* memory: direct */
/* safer: write */
/* preload: random */
/* speedcheck: no */
/* alloc: dynamic */
```

Listing 8: SB.litmus

```
%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%
% Results for SB+fen.litmus %
%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%
AArch64 SB+fen
"DMB.SYdWR Fre DMB.SYdWR Fre"

{
 0:X1=x; 0:X2=y;
 1:X1=y; 1:X2=x;
}
 P0          | P1          ;
 MOV W0,#1   | MOV W0,#1   ;
 STR W0,[X1] | STR W0,[X1] ;
 DMB SY      | DMB SY      ;
 LDR W3,[X2] | LDR W3,[X2] ;

exists (0:X3=0 /\ 1:X3=0)
Generated assembler
        ;START _litmus_P0
        mov     w0, #1                          ; =0x1
        str     w0, [x15]
        dmb     sy
        ldr     w17, [x16]
        ;START _litmus_P1
        mov     w0, #1                          ; =0x1
        str     w0, [x15]
        dmb     sy
        ldr     w17, [x16]

Test SB+fen Allowed
Histogram (3 states)
49999700:>0:X3=1; 1:X3=0;
49999830:>0:X3=0; 1:X3=1;
470   :>0:X3=1; 1:X3=1;
No

Witnesses
Positive: 0, Negative: 100000000
Condition exists (0:X3=0 /\ 1:X3=0) is NOT validated
Hash=c6a533fb255005c1a89998321e552a86
Generator=diyone7 (version 7.58)
Com=Fr Fr
```

```
Orig=DMB.SYdWR Fre DMB.SYdWR Fre
Observation SB+fen Never 0 100000000
Time SB+fen 9.41

Machine:Kyles-MacBook-Pro.local
Hardware:

    Hardware Overview:

      Model Name: MacBook Pro
      Model Identifier: Mac16,5
      Model Number: MX303B/A
      Chip: Apple M4 Max
      Total Number of Cores: 14 (10 performance and 4 efficiency)
      Memory: 36 GB
      System Firmware Version: 11881.81.4
      OS Loader Version: 11881.81.4
      Serial Number (system): C00C3JNQFV
      Hardware UUID: 6B16ED3A-820B-5B1F-A79E-0B45691B2525
      Provisioning UDID: 00006041-000849C90204801C
      Activation Lock Status: Enabled

Revision exported, version 7.58
Command line: litmus7 SB+fen.litmus -s 1k -r 100k
Parameters
#define SIZE_OF_TEST 1000
#define NUMBER_OF_RUN 100000
#define AVAIL 1
#define STRIDE (-1)
#define MAX_LOOP 0
/* gcc options: -Wall -std=gnu99 -O2 */
/* barrier: user */
/* launch: changing */
/* affinity: none */
/* memory: direct */
/* safer: write */
/* preload: random */
/* speedcheck: no */
/* alloc: dynamic */
```

Listing 9: SB3.litmus

```
%%%%%%%%%%%%%%%%%%%%%%%%%%
% Results for SB3.litmus %
```

```
%%%%%%%%%%%%%%%%%%%%%%%%%
AArch64 SB3
"PodWR Fre PodWR Fre PodWR Fre"

{
 0:X1=x; 0:X2=y;
 1:X1=y; 1:X2=z;
 2:X1=z; 2:X2=x;
}
 P0          | P1          | P2          ;
 MOV W0,#1   | MOV W0,#1   | MOV W0,#1   ;
 STR W0,[X1] | STR W0,[X1] | STR W0,[X1] ;
 LDR W3,[X2] | LDR W3,[X2] | LDR W3,[X2] ;

exists (0:X3=0 /\ 1:X3=0 /\ 2:X3=0)
Generated assembler
        ;START _litmus_P0
        mov     w1, #1                          ; =0x1
        str     w1, [x16]
        ldr     w0, [x17]
        ;START _litmus_P1
        mov     w1, #1                          ; =0x1
        str     w1, [x16]
        ldr     w0, [x17]
        ;START _litmus_P2
        mov     w1, #1                          ; =0x1
        str     w1, [x16]
        ldr     w0, [x17]

Test SB3 Allowed
Histogram (8 states)
1870409*>0:X3=0; 1:X3=0; 2:X3=0;
22261816:>0:X3=1; 1:X3=0; 2:X3=0;
22964902:>0:X3=0; 1:X3=1; 2:X3=0;
9786516:>0:X3=1; 1:X3=1; 2:X3=0;
22648536:>0:X3=0; 1:X3=0; 2:X3=1;
10249446:>0:X3=1; 1:X3=0; 2:X3=1;
10218362:>0:X3=0; 1:X3=1; 2:X3=1;
13    :>0:X3=1; 1:X3=1; 2:X3=1;
Ok

Witnesses
Positive: 1870409, Negative: 98129591
Condition exists (0:X3=0 /\ 1:X3=0 /\ 2:X3=0) is validated
```

```
Hash=538871abd7556401017a12c13ec7d468
Generator=diyone7 (version 7.58)
Com=Fr Fr Fr
Orig=PodWR Fre PodWR Fre PodWR Fre
Observation SB3 Sometimes 1870409 98129591
Time SB3 10.40

Machine:Kyles-MacBook-Pro.local
Hardware:

    Hardware Overview:

      Model Name: MacBook Pro
      Model Identifier: Mac16,5
      Model Number: MX303B/A
      Chip: Apple M4 Max
      Total Number of Cores: 14 (10 performance and 4 efficiency)
      Memory: 36 GB
      System Firmware Version: 11881.81.4
      OS Loader Version: 11881.81.4
      Serial Number (system): C00C3JNQFV
      Hardware UUID: 6B16ED3A-820B-5B1F-A79E-0B45691B2525
      Provisioning UDID: 00006041-000849C90204801C
      Activation Lock Status: Enabled

Revision exported, version 7.58
Command line: litmus7 SB3.litmus -s 1k -r 100k
Parameters
#define SIZE_OF_TEST 1000
#define NUMBER_OF_RUN 100000
#define AVAIL 1
#define STRIDE (-1)
#define MAX_LOOP 0
/* gcc options: -Wall -std=gnu99 -O2 */
/* barrier: user */
/* launch: changing */
/* affinity: none */
/* memory: direct */
/* safer: write */
/* preload: random */
/* speedcheck: no */
/* alloc: dynamic */
```

Listing 10: SB3.litmus

```
%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%
% Results for SB3+fen.litmus %
%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%
AArch64 SB3+fen
"DMB.SYdWR Fre DMB.SYdWR Fre DMB.SYdWR Fre"

{
 0:X1=x; 0:X2=y;
 1:X1=y; 1:X2=z;
 2:X1=z; 2:X2=x;
}
 P0          | P1          | P2          ;
 MOV W0,#1   | MOV W0,#1   | MOV W0,#1   ;
 STR W0,[X1] | STR W0,[X1] | STR W0,[X1] ;
 DMB SY      | DMB SY      | DMB SY      ;
 LDR W3,[X2] | LDR W3,[X2] | LDR W3,[X2] ;

exists (0:X3=0 /\ 1:X3=0 /\ 2:X3=0)
Generated assembler
        ;START _litmus_P0
        mov     w1, #1                          ; =0x1
        str     w1, [x16]
        dmb     sy
        ldr     w0, [x17]
        ;START _litmus_P1
        mov     w1, #1                          ; =0x1
        str     w1, [x16]
        dmb     sy
        ldr     w0, [x17]
        ;START _litmus_P2
        mov     w1, #1                          ; =0x1
        str     w1, [x16]
        dmb     sy
        ldr     w0, [x17]

Test SB3+fen Allowed
Histogram (7 states)
14724712:>0:X3=1; 1:X3=0; 2:X3=0;
15172160:>0:X3=0; 1:X3=1; 2:X3=0;
18158465:>0:X3=1; 1:X3=1; 2:X3=0;
15706702:>0:X3=0; 1:X3=0; 2:X3=1;
18607962:>0:X3=1; 1:X3=0; 2:X3=1;
17627836:>0:X3=0; 1:X3=1; 2:X3=1;
```

```
2163  :>0:X3=1; 1:X3=1; 2:X3=1;
No

Witnesses
Positive: 0, Negative: 100000000
Condition exists (0:X3=0 /\ 1:X3=0 /\ 2:X3=0) is NOT validated
Hash=af8ef14ca36cdca9dcec373504dd21e5
Generator=diyone7 (version 7.58)
Com=Fr Fr Fr
Orig=DMB.SYdWR Fre DMB.SYdWR Fre DMB.SYdWR Fre
Observation SB3+fen Never 0 100000000
Time SB3+fen 12.40

Machine:Kyles-MacBook-Pro.local
Hardware:

    Hardware Overview:

      Model Name: MacBook Pro
      Model Identifier: Mac16,5
      Model Number: MX303B/A
      Chip: Apple M4 Max
      Total Number of Cores: 14 (10 performance and 4 efficiency)
      Memory: 36 GB
      System Firmware Version: 11881.81.4
      OS Loader Version: 11881.81.4
      Serial Number (system): C00C3JNQFV
      Hardware UUID: 6B16ED3A-820B-5B1F-A79E-0B45691B2525
      Provisioning UDID: 00006041-000849C90204801C
      Activation Lock Status: Enabled

Revision exported, version 7.58
Command line: litmus7 SB3+fen.litmus -s 1k -r 100k
Parameters
#define SIZE_OF_TEST 1000
#define NUMBER_OF_RUN 100000
#define AVAIL 1
#define STRIDE (-1)
#define MAX_LOOP 0
/* gcc options: -Wall -std=gnu99 -O2 */
/* barrier: user */
/* launch: changing */
/* affinity: none */
/* memory: direct */
```

```
/* safer: write */
/* preload: random */
/* speedcheck: no */
/* alloc: dynamic */
```

1.c herd7

Listing 11: SB.litmus

```
Test SB Allowed
States 4
0:X3=0; 1:X3=0;
0:X3=0; 1:X3=1;
0:X3=1; 1:X3=0;
0:X3=1; 1:X3=1;
Ok
Witnesses
Positive: 1 Negative: 3
Condition exists (0:X3=0 /\ 1:X3=0)
Observation SB Sometimes 1 3
Time SB 0.01
Hash=0e1eb22a42a7de6af162a7e5cbb4e4d3
```

Listing 12: SB.litmus

```
Test SB3 Allowed
States 8
0:X3=0; 1:X3=0; 2:X3=0;
0:X3=0; 1:X3=0; 2:X3=1;
0:X3=0; 1:X3=1; 2:X3=0;
0:X3=0; 1:X3=1; 2:X3=1;
0:X3=1; 1:X3=0; 2:X3=0;
0:X3=1; 1:X3=0; 2:X3=1;
0:X3=1; 1:X3=1; 2:X3=0;
0:X3=1; 1:X3=1; 2:X3=1;
Ok
Witnesses
Positive: 1 Negative: 7
Condition exists (0:X3=0 /\ 1:X3=0 /\ 2:X3=0)
Observation SB3 Sometimes 1 7
Time SB3 0.01
Hash=538871abd7556401017a12c13ec7d468
```

Listing 13: SB3.litmus

```
Test SB+fen Allowed
States 3
0:X3=0; 1:X3=1;
0:X3=1; 1:X3=0;
0:X3=1; 1:X3=1;
No
Witnesses
Positive: 0 Negative: 3
Condition exists (0:X3=0 /\ 1:X3=0)
Observation SB+fen Never 0 3
Time SB+fen 0.01
Hash=c6a533fb255005c1a89998321e552a86
```

Listing 14: SB3.litmus

```
Test SB3+fen Allowed
States 7
0:X3=0; 1:X3=0; 2:X3=1;
0:X3=0; 1:X3=1; 2:X3=0;
0:X3=0; 1:X3=1; 2:X3=1;
0:X3=1; 1:X3=0; 2:X3=0;
0:X3=1; 1:X3=0; 2:X3=1;
0:X3=1; 1:X3=1; 2:X3=0;
0:X3=1; 1:X3=1; 2:X3=1;
No
Witnesses
Positive: 0 Negative: 7
Condition exists (0:X3=0 /\ 1:X3=0 /\ 2:X3=0)
Observation SB3+fen Never 0 7
Time SB3+fen 0.01
Hash=af8ef14ca36cdca9dcec373504dd21e5
```

1.d rmem

Listing 15: SB.litmus

```
Test SB Allowed
Memory_writes=
States 4
2     *>0:X3=0; 1:X3=0;  via "1;2;3;0"
3     :>0:X3=0; 1:X3=1;  via "0;1;2;3"
3     :>0:X3=1; 1:X3=0;  via "2;1;3;0"
4     :>0:X3=1; 1:X3=1;  via "0;2;1;3"
```

```
Unhandled exceptions 0
Ok
Condition exists (0:X3=0 /\ 1:X3=0)
Hash=0e1eb22a42a7de6af162a7e5cbb4e4d3
Observation SB Sometimes 1 3
Runtime: 0.116763 sec
```

Listing 16: SB.litmus

```
Test SB+fen Allowed
Memory-writes=
States 3
1     :>0:X3=0; 1:X3=1;  via "0;0;1;0"
1     :>0:X3=1; 1:X3=0;  via "1;1;0;0"
4     :>0:X3=1; 1:X3=1;  via "0;1;0;1"
Unhandled exceptions 0

No (allowed not found)
Condition exists (0:X3=0 /\ 1:X3=0)
Hash=c6a533fb255005c1a89998321e552a86
Observation SB+fen Never 0 3
Runtime: 0.090825 sec
```

Listing 17: SB3.litmus

```
Test SB3 Allowed
Memory-writes=
States 8
6     *>0:X3=0; 1:X3=0; 2:X3=0;  via "1;2;3;4;5;0"
10    :>0:X3=0; 1:X3=0; 2:X3=1;  via "0;1;2;3;4;5"
10    :>0:X3=0; 1:X3=1; 2:X3=0;  via "1;2;4;3;5;0"
14    :>0:X3=0; 1:X3=1; 2:X3=1;  via "0;1;2;4;3;5"
10    :>0:X3=1; 1:X3=0; 2:X3=0;  via "2;1;3;4;5;0"
14    :>0:X3=1; 1:X3=0; 2:X3=1;  via "0;2;1;3;4;5"
14    :>0:X3=1; 1:X3=1; 2:X3=0;  via "2;1;4;3;5;0"
18    :>0:X3=1; 1:X3=1; 2:X3=1;  via "0;2;1;4;3;5"
Unhandled exceptions 0
Ok
Condition exists (0:X3=0 /\ 1:X3=0 /\ 2:X3=0)
Hash=538871abd7556401017a12c13ec7d468
Observation SB3 Sometimes 1 7
Runtime: 0.735628 sec
```

Listing 18: SB3.litmus

```
Test SB3+fen Allowed
Memory-writes=
States 7
1     :>0:X3=0; 1:X3=0; 2:X3=1;  via "0;0;1;0;2;0"
1     :>0:X3=0; 1:X3=1; 2:X3=0;  via "2;2;0;0;1;0"
6     :>0:X3=0; 1:X3=1; 2:X3=1;  via "0;0;1;2;0;1"
1     :>0:X3=1; 1:X3=0; 2:X3=0;  via "1;1;2;1;0;0"
6     :>0:X3=1; 1:X3=0; 2:X3=1;  via "0;1;0;1;2;0"
6     :>0:X3=1; 1:X3=1; 2:X3=0;  via "1;2;1;2;0;0"
18    :>0:X3=1; 1:X3=1; 2:X3=1;  via "0;1;0;2;1;0"
Unhandled exceptions 0

No (allowed not found)
Condition exists (0:X3=0 /\ 1:X3=0 /\ 2:X3=0)
Hash=af8ef14ca36cdca9dcec373504dd21e5
Observation SB3+fen Never 0 7
Runtime: 0.322971 sec
```

1.e mcompare7

Listing 19: SB.litmus

```
*Diffs*
  |Kind | b_1.txt           c_1.txt
-------------------------------------
-------------------------------------
SB|Allow| [0:X3=0; 1:X3=0;] ==
  |Ok   | [0:X3=0; 1:X3=1;]
  |     | [0:X3=1; 1:X3=0;]
  |     | [0:X3=1; 1:X3=1;]
```

Listing 20: SB.litmus

```
*Diffs*
      |Kind | b_1fen.txt        c_1fen.txt
-----------------------------------------------
-----------------------------------------------
SB+fen|Allow| [0:X3=0; 1:X3=1;] ---
      |No   | [0:X3=1; 1:X3=0;]
      |     | [0:X3=1; 1:X3=1;]
```

Listing 21: SB3.litmus

```
*Diffs*
```

```
   |Kind | b_2.txt                    c_2.txt
-----------------------------------------------
-----------------------------------------------
SB3|Allow| [0:X3=0; 1:X3=0; 2:X3=0;] ---
   |Ok   | [0:X3=0; 1:X3=0; 2:X3=1;]
   |     | [0:X3=0; 1:X3=1; 2:X3=0;]
   |     | [0:X3=0; 1:X3=1; 2:X3=1;]
   |     | [0:X3=1; 1:X3=0; 2:X3=0;]
   |     | [0:X3=1; 1:X3=0; 2:X3=1;]
   |     | [0:X3=1; 1:X3=1; 2:X3=0;]
   |     | [0:X3=1; 1:X3=1; 2:X3=1;]
```

Listing 22: SB3.litmus

```
*Diffs*
       |Kind | b_2fen.txt                 c_2fen.txt
-------------------------------------------------------
-------------------------------------------------------
SB3+fen|Allow| [0:X3=0; 1:X3=0; 2:X3=1;] ==
       |No   | [0:X3=0; 1:X3=1; 2:X3=0;]
       |     | [0:X3=0; 1:X3=1; 2:X3=1;]
       |     | [0:X3=1; 1:X3=0; 2:X3=0;]
       |     | [0:X3=1; 1:X3=0; 2:X3=1;]
       |     | [0:X3=1; 1:X3=1; 2:X3=0;]
       |     | [0:X3=1; 1:X3=1; 2:X3=1;]
```

2 Appendix

2.a

Listing 23: sub.litmus

```
Test S+fen+data-wsi Allowed
States 3
1:X1=0; 1:X3=0;
1:X1=0; 1:X3=1;
1:X1=1; 1:X3=1;
No
Witnesses
Positive: 0 Negative: 3
Condition exists (1:X1=1 /\ 1:X3=0)
Observation S+fen+data-wsi Never 0 3
Time S+fen+data-wsi 0.01
Hash=6cd45049c39f769e3371a253e6fa3e5c
```

,,

Listing 24: litmus7 sub.litmus with 1e8 runs output

```
Test S+fen+data-wsi Allowed
States 3
1:X1=0; 1:X3=0;
1:X1=0; 1:X3=1;
1:X1=1; 1:X3=1;
No
Witnesses
Positive: 0 Negative: 3
Condition exists (1:X1=1 /\ 1:X3=0)
Observation S+fen+data-wsi Never 0 3
Time S+fen+data-wsi 0.01
Hash=6cd45049c39f769e3371a253e6fa3e5c
```


UNIVERSITY OF CAMBRIDGE

3 Appendix

This includes the compiled code for the Message Passing program.

Listing 25: Compiled MessagePassing

```
writer:
        sub     sp, sp, #32
        str     x0, [sp, #24]
        ldr     x8, [sp, #24]
        str     x8, [sp, #16]
        ldr     x8, [sp, #16]
        ldr     x9, [x8]
        mov     w8, #1
        str     w8, [x9]
        ldr     x9, [sp, #16]
        ldr     x9, [x9, #8]
        str     w8, [sp, #12]
        ldr     w8, [sp, #12]
        stlr    w8, [x9]
        mov     x0, xzr
        add     sp, sp, #32
        ret

reader:
        sub     sp, sp, #48
        stp     x29, x30, [sp, #32]
        add     x29, sp, #32
        stur    x0, [x29, #-8]
        ldur    x8, [x29, #-8]
        str     x8, [sp, #16]
        b       .LBB1_1
.LBB1_1:
        ldr     x8, [sp, #16]
        ldr     x8, [x8, #8]
        ldar    w8, [x8]
        str     w8, [sp, #12]
        ldr     w8, [sp, #12]
        cbnz    w8, .LBB1_3
        b       .LBB1_2
.LBB1_2:
        b       .LBB1_1
.LBB1_3:
        ldr     x8, [sp, #16]
```

```
        ldr     x8, [x8]
        ldr     w1, [x8]
        adrp    x0, .L.str
        add     x0, x0, :lo12:.L.str
        bl      printf
        mov     x0, xzr
        ldp     x29, x30, [sp, #32]
        add     sp, sp, #48
        ret

main:
        sub     sp, sp, #96
        stp     x29, x30, [sp, #80]
        add     x29, sp, #80
        mov     w8, wzr
        str     w8, [sp, #28]
        stur    wzr, [x29, #-4]
        sub     x9, x29, #28
        stur    wzr, [x29, #-28]
        sub     x8, x29, #32
        stur    wzr, [x29, #-32]
        add     x3, sp, #32
        str     x3, [sp, #8]
        str     x9, [sp, #32]
        str     x8, [sp, #40]
        sub     x0, x29, #16
        mov     x1, xzr
        str     x1, [sp, #16]
        adrp    x2, writer
        add     x2, x2, :lo12:writer
        bl      pthread_create
        ldr     x3, [sp, #8]
        ldr     x1, [sp, #16]
        sub     x0, x29, #24
        adrp    x2, reader
        add     x2, x2, :lo12:reader
        bl      pthread_create
        ldr     x1, [sp, #16]
        ldur    x0, [x29, #-16]
        bl      pthread_join
        ldr     x1, [sp, #16]
        ldur    x0, [x29, #-24]
        bl      pthread_join
        ldr     w0, [sp, #28]
```

```
        ldp     x29, x30, [sp, #80]
        add     sp, sp, #96
        ret

.L.str:
        .asciz  "read x: %d\n"
```


UNIVERSITY OF CAMBRIDGE

References

[1] J. Alglave, L. Maranget, and M. Tautschnig, "Herding cats: Modelling, declarative testing, and empirical evaluation of weak memory," in *ACM SIGPLAN Notices*, vol. 49, pp. 7–17, ACM, 2014.

[2] T. L. Harris, "A pragmatic implementation of non-blocking linked-lists," in *International Symposium on Distributed Computing*, pp. 300–314, Springer, 2001.

[3] A. Braginsky, A. Kogan, and E. Petrank, "Drop the anchor: lightweight memory management for non-blocking data structures," in *Proceedings of the Twenty-Fifth Annual ACM Symposium on Parallelism in Algorithms and Architectures*, SPAA '13, (New York, NY, USA), p. 33–42, Association for Computing Machinery, 2013.

List of Figures

List of Tables